

Lecture 2

COMP2211: Distributed Systems

Middleware Technologies and Web Services

Thiru Vasantam

Lecture 2

- Recap:
 - Introduction to distributed systems: properties, benefits
 - Transparency: different types
 - Design challenges
- Today:
 - Topic 2: Middleware
 - Remote Procedure Call (RPC)
 - Remote Method Invocation (RMI)
 - Message Oriented Middleware (MOM)
 - Topic 3: Web Services

Topic2: Middleware Technologies

Aim

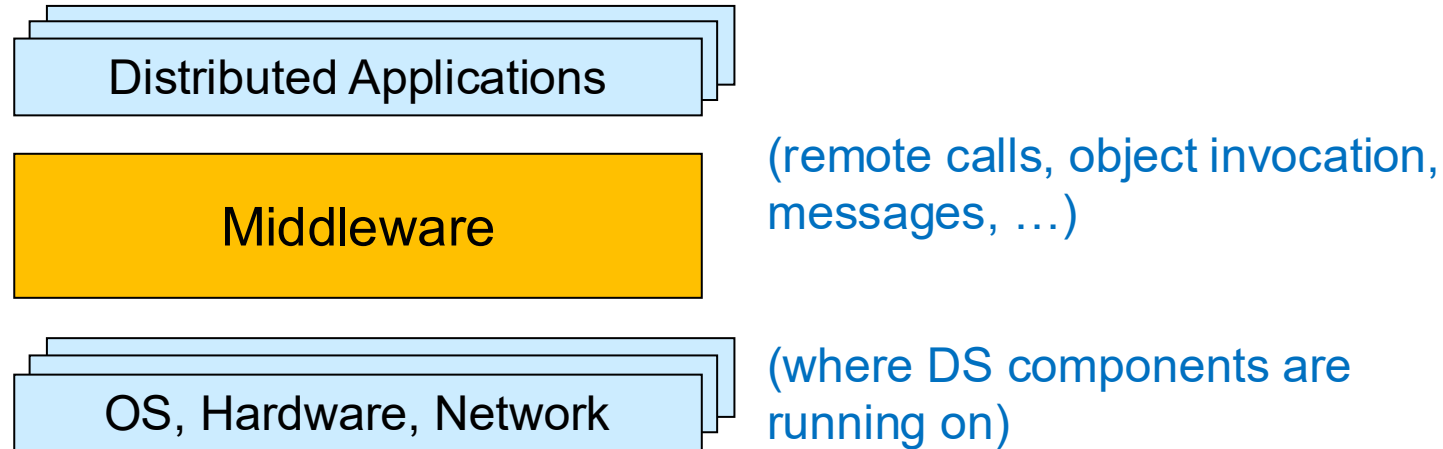
Study Middleware technologies

Understand how they support distributed systems development

Study RPC, RMI, MOM

Middleware Technologies

- Software technology that enables distributed system (DS) components to work together and to communicate as if they were virtually non-distributed.



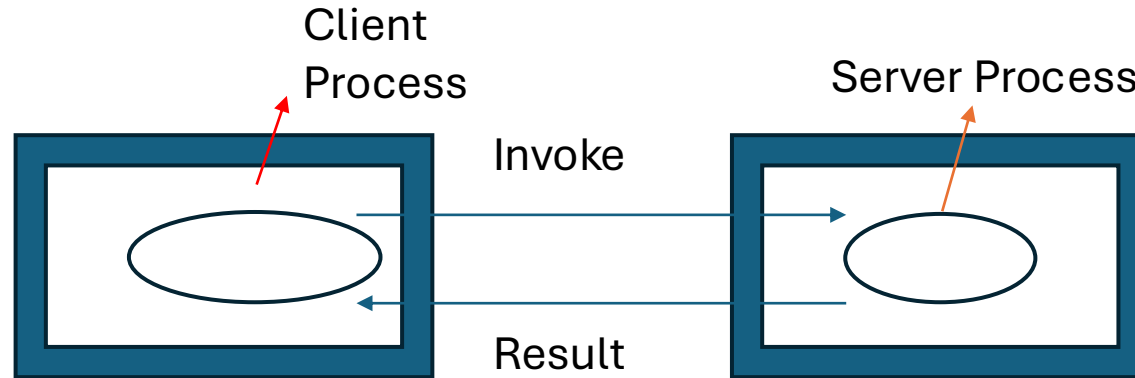
- User perspective:
 - Hide users from the implementation details and the “distributed” nature
- Developer perspective:
 - Hide DS developers from low-level networking details
 - Provide common programming abstraction and infrastructure for constructing distributed applications

Develop DS with Middleware

Aims:

- Focus on application logics
- Save efforts in implementing socket communication and dealing with network architecture and its dynamic changes
- Make distributed computing look like centralised computing

Develop DS with Middleware



Middleware options:

- Remote Procedure Call (RPC)
 - Inter-process communication; execute a remote function without the programmer coding the network communication; uses C language
- Object-Oriented Middleware (OOM)
 - Extend the idea of RPC to allow remote invocation of objects; uses Java/Python
- Message-Oriented Middleware (MOM)
- Web Services

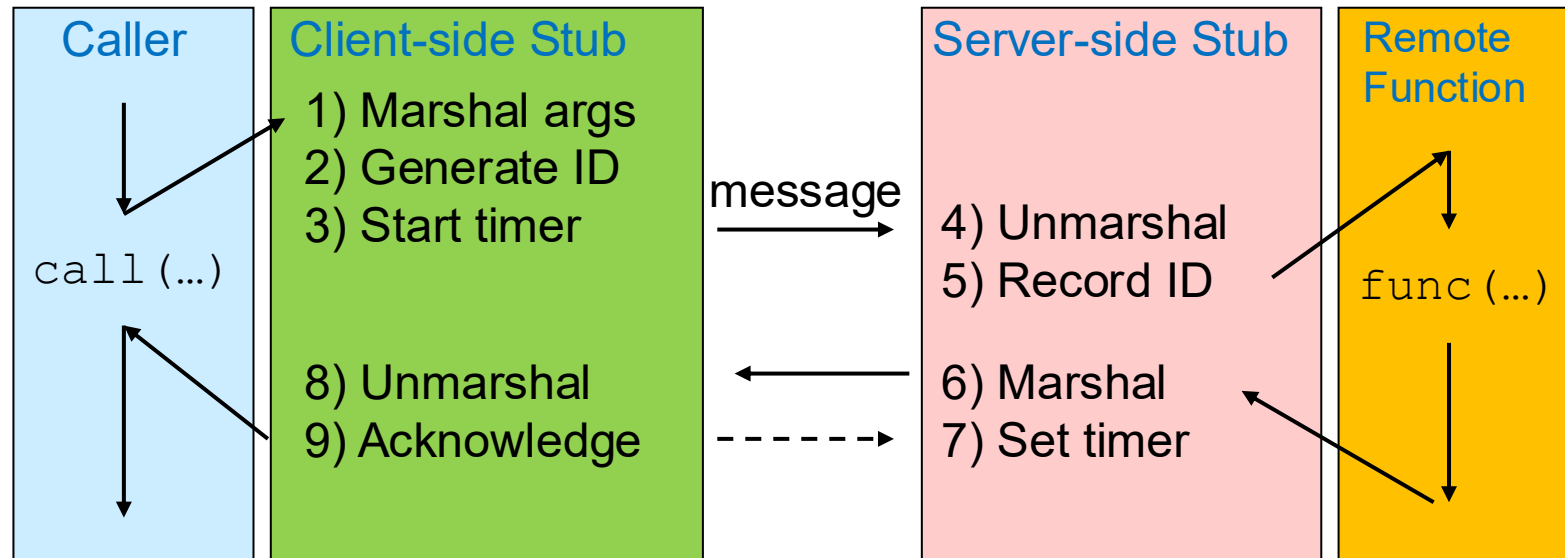
Remote Procedure Calls

Goal: Make distributed computing look like centralised computing

- Allow remote services to be called as procedures
 - Transparency with regard to location, implementation, and language
- Issues
 - How to pass parameters
 - Bindings
 - Semantics in the face of errors

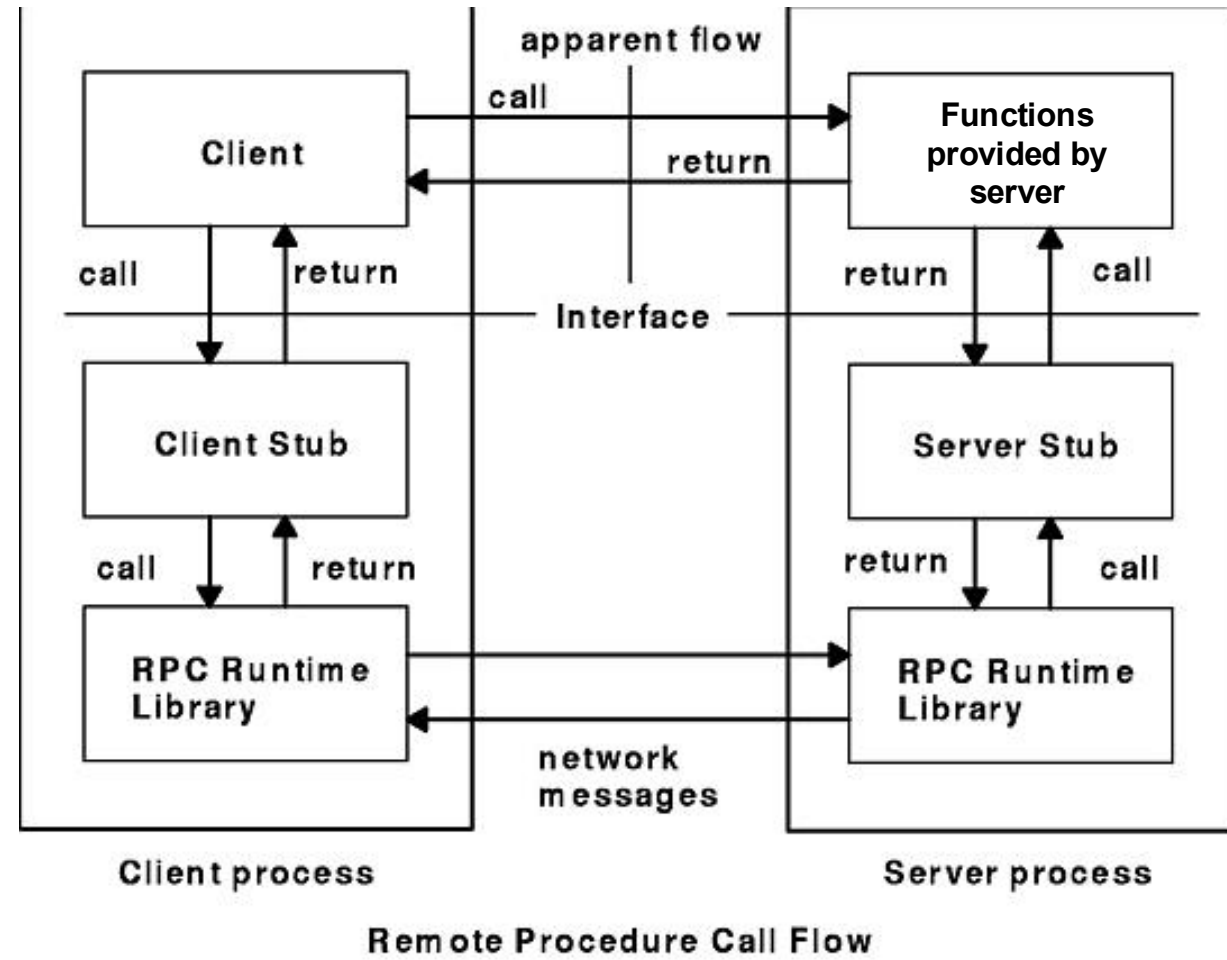
Remote Procedure Call (RPC)

- Adapt the traditional program paradigm, dividing a system into procedures, i.e., subroutines/functions
- Mask remote function calls as being local, supporting distributed development



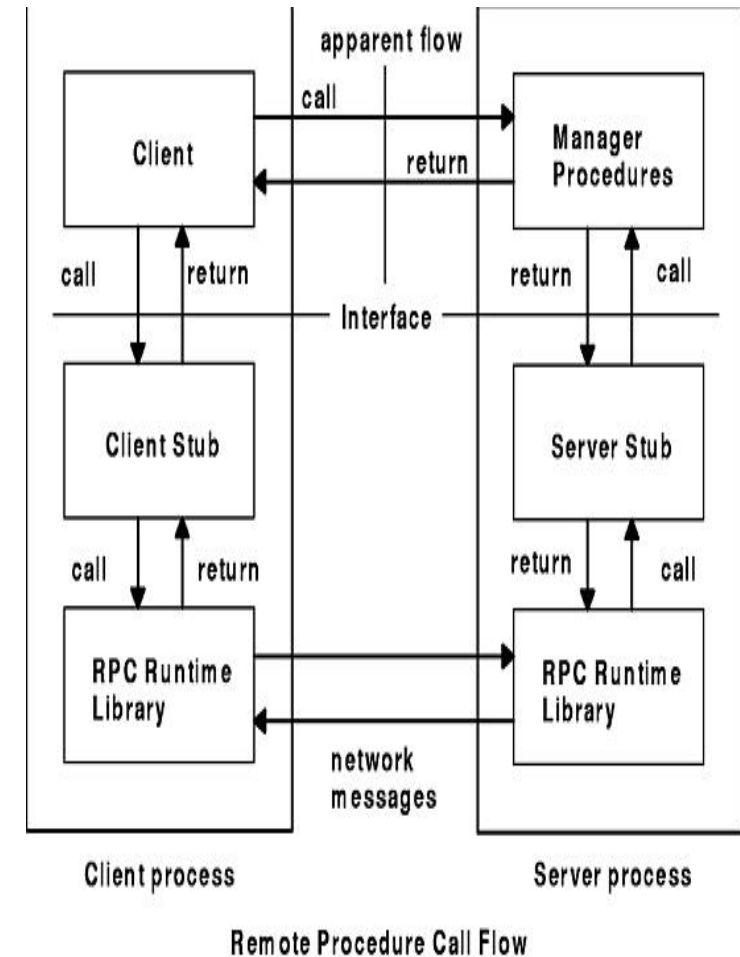
- The request/reply paradigm is usually implemented with message passing in an RPC service
- Marshalling of function parameters and return value

How RPC Works



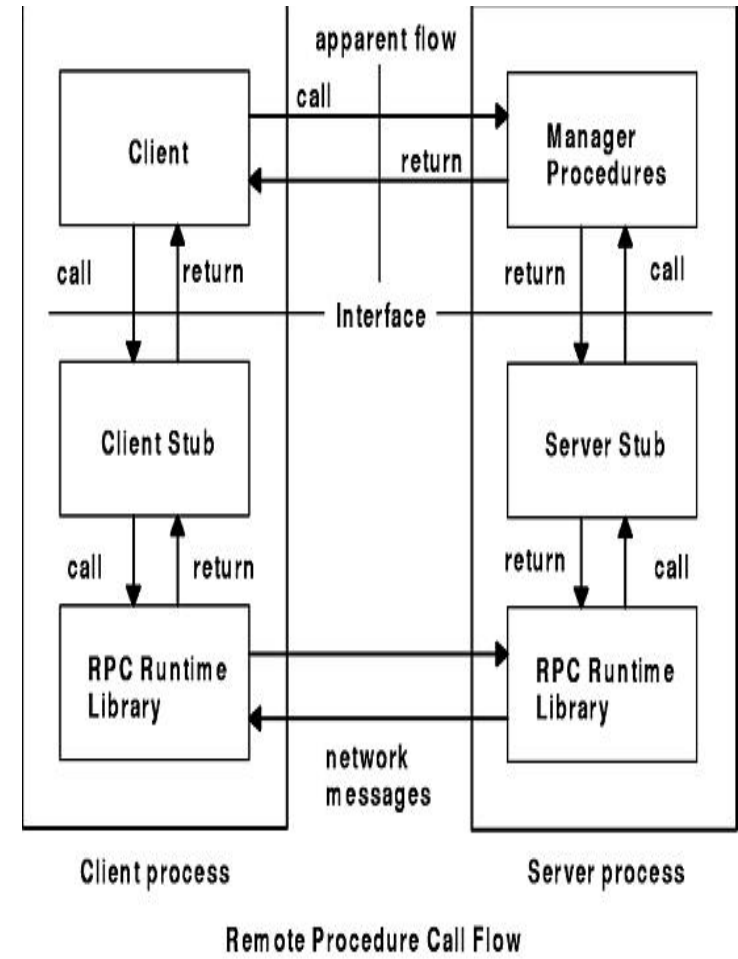
RPC Program Development

- **Server:** defines the service interface using an *interface definition language* (IDL), which specifies names, parameters, and types for all client-callable procedures
- **Stub compiler:** reads the IDL declarations and produces two *stub functions* for each server function
 - *Server-side* and *client-side*
- **Linking:** Server programmer implements the service's functions and links with the *server-side* stubs; Client programmer implements the client program and links with the *client-side* stubs

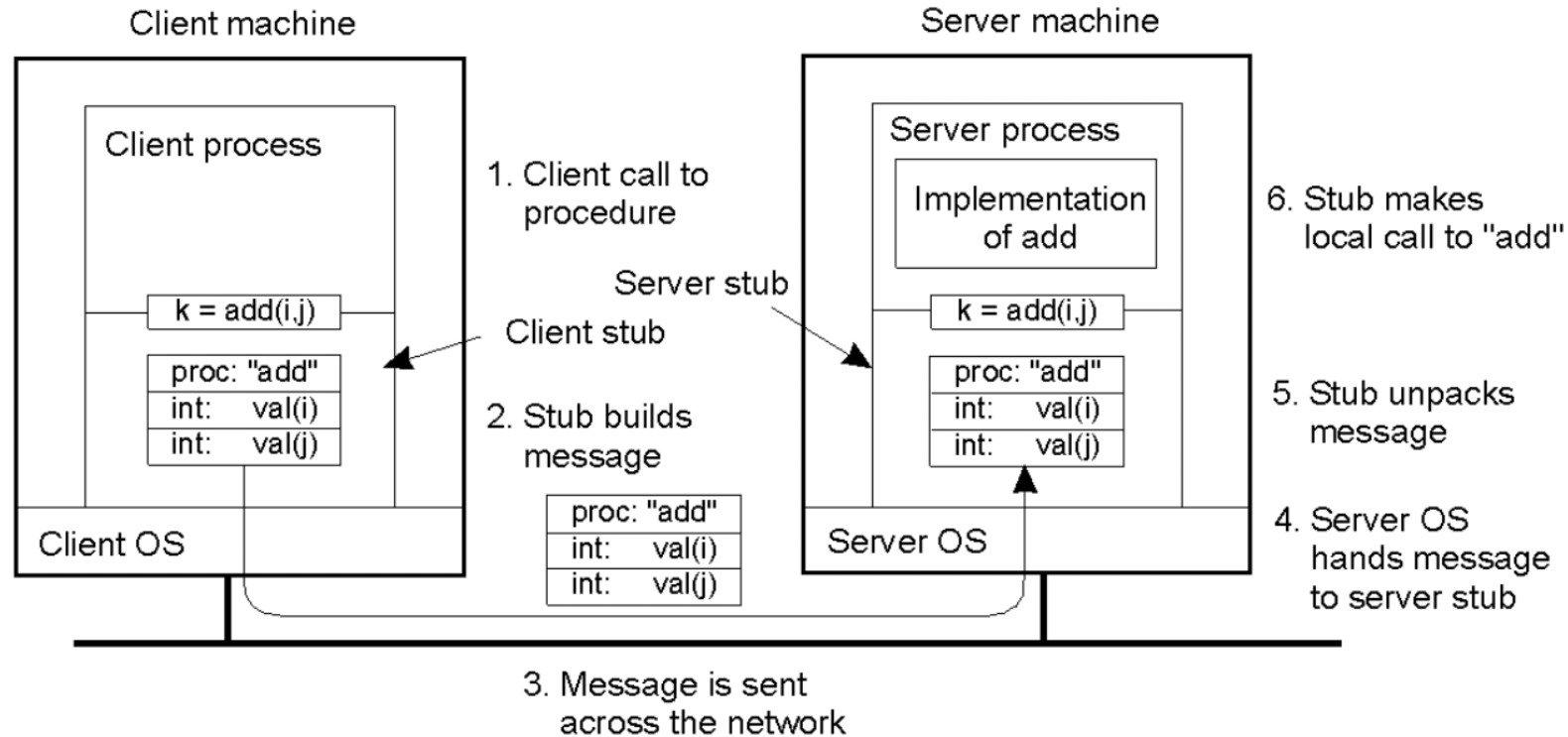


RPC Program Development

- **Operation:** Stubs manage all of the details of remote communication between client and server
- **RPC Runtime Library:** Contains tables, data, and routines required for communication between client and server
 - Hides the encoding/decoding of parameters and results and the passing of messages from the application programmer



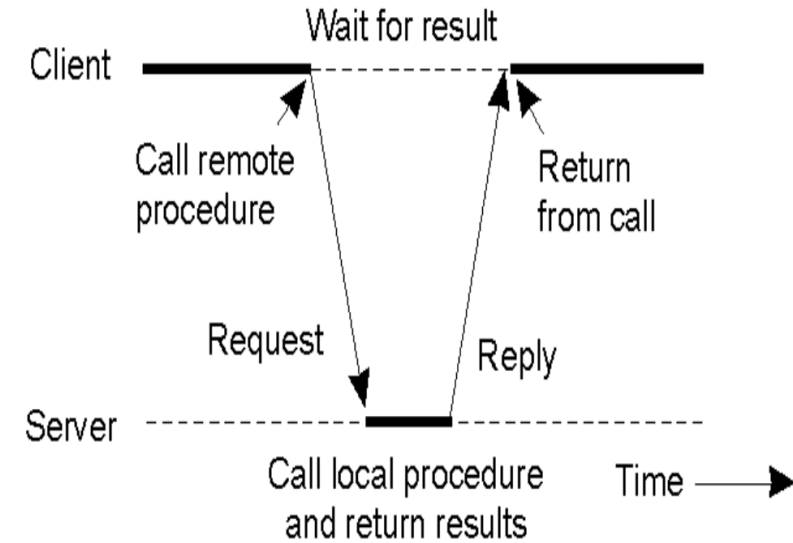
An Example of RPC



Source: Steen & Tannenbaum textbook

Properties and Limitations

- Synchronous request/reply interaction
 - Holds a connection open while waiting for the response to arrive or for the timeout to expire.
 - Tight coupling between client and server.
 - The client may be blocked for a long time if the server is overloaded
 - Slow/failed clients may delay servers



Source: Steen & Tannenbaum textbook

Can you identify any disadvantages of this method?

Answer: The performance of the system is bad, as client/server wait unnecessarily

Properties and Limitations: Parameter Passing

- Local procedure parameter passing
 - Call-by-value
 - Call-by-reference: arrays, complex data structures
- Remote procedure calls are implemented through stubs:
 - Global variables are not allowed in RPCs
 - Only Call-by-value is allowed

Properties and Limitations

- Host information is required
 - Location transparency cannot be facilitated
- ```
clnt_stat = callrpc (host, prog_ID, prog_ver,
*arguments, in_xdr_filter, *result, out_xdr_filter);
```

callrpc calls the remote procedure mentioned in the parameters; host is the name of the server

# Properties and Limitations

- Programme paradigm: not object-oriented
  - invoke functions on servers: no encapsulation or inheritance support
    - Encapsulation means control of access to procedures, like public/private procedures
    - Inheritance means that some functionalities of a procedure are inherited by another procedure

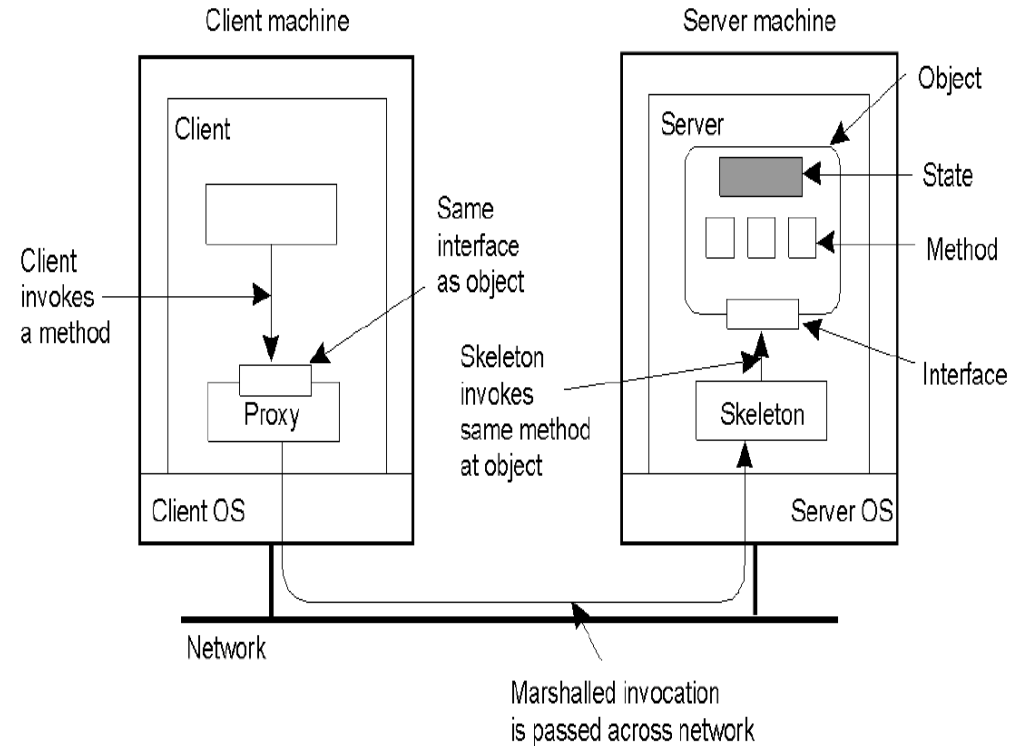


# Remote Method Invocation (RMI)

- RPCs applied to *objects*, i.e., instances of a class
  - *Class*: object-oriented abstraction; module with data and operations
  - Separation between interface and implementation
  - Interface resides on one machine, implementation on another
- RMIs support system-wide object references
  - Parameters can be object references

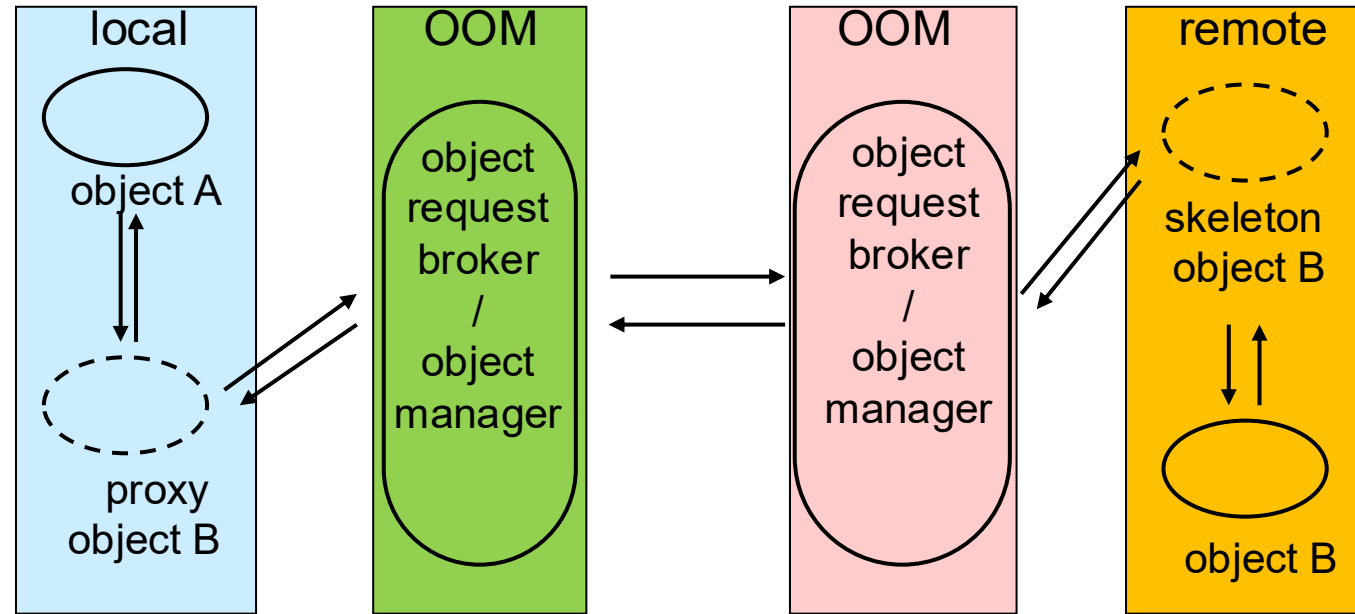
# Distributed Objects

- When a client binds to a distributed object, load the interface (“proxy”) into the client address space
  - Proxy is analogous to stubs
- Server stub is referred to as a skeleton



Source: Steen & Tannenbaum textbook

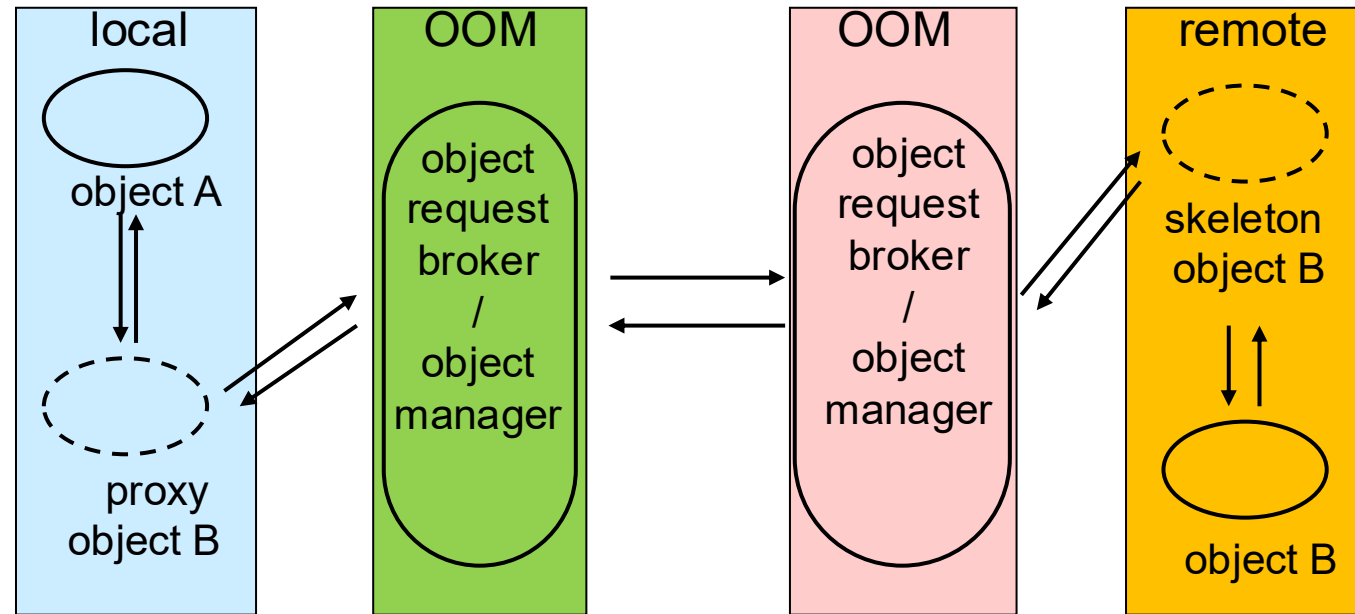
# Object-Oriented Middleware (OOM)



**Objects** contain a set of data and methods, can be *local* or *remote*, and they interact with other objects, **Ex:** Java, Python

- Remote objects are visible through **remote interfaces**
- RMI masks remote objects as being local using **proxy objects/stubs**
- Skeleton object directs incoming calls from clients to the appropriate object

# Object-Oriented Middleware (OOM)

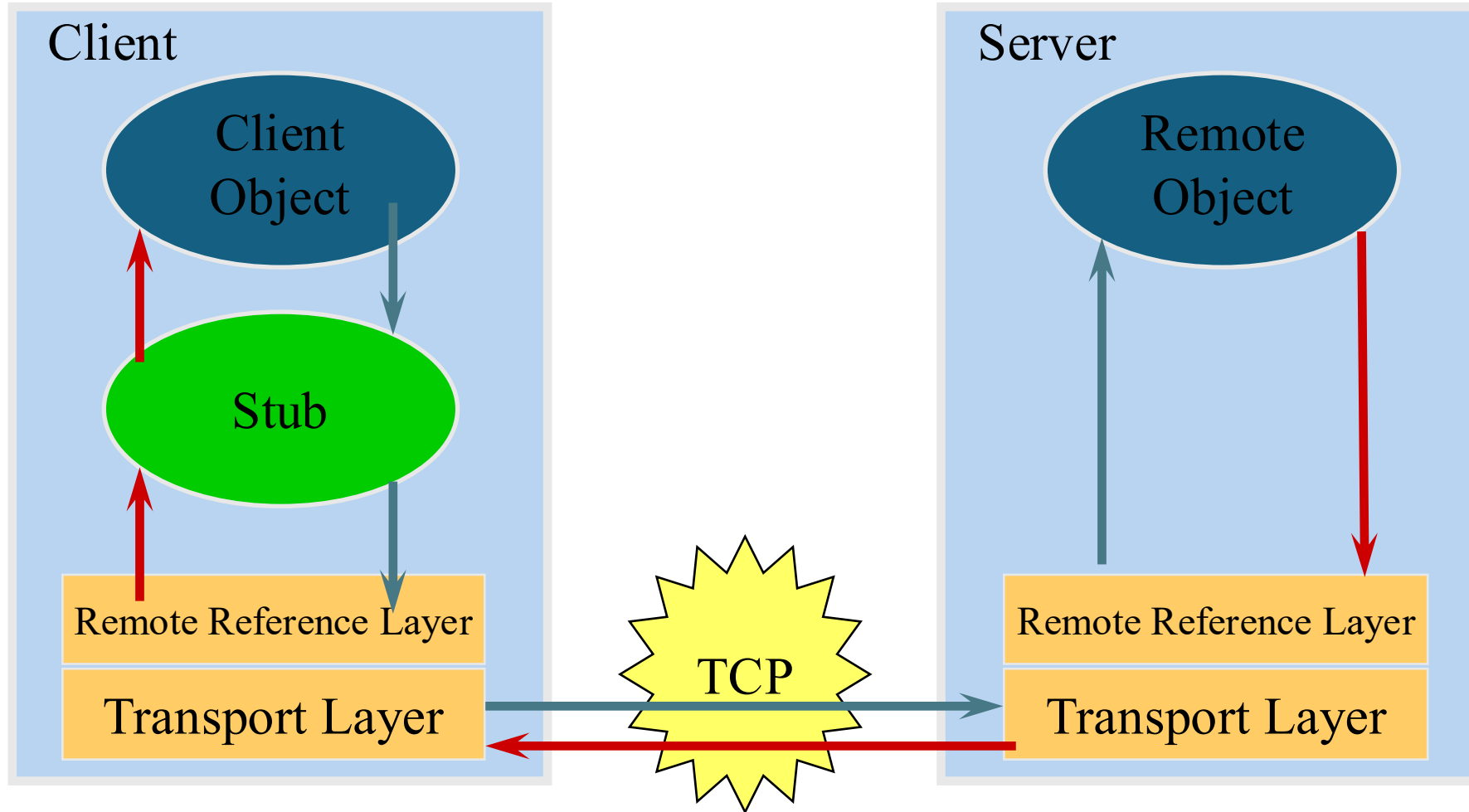


- **Object request broker** identifies / discovers remote objects
- Uses object references to call remote objects

# Java RMI and Pyro

- Remote Method Invocation (RMI) is originated from Java to allow object-to-object communication among Java objects for realising a distributed system.
- RMI allows us to distribute our objects on various machines and invoke methods on the objects located on remote sites.
- **Advantage:** Dynamically invoke new versions of remote objects
- **Application: Utilise** very fast remote processors or any specialised resources
- Reference: <http://docs.oracle.com/javase/tutorial/rmi/>
- Pyro: Python implementation of Java RMI
- Reference: <https://pypi.org/project/Pyro4/> or <https://pypi.org/project/Pyro5/>

# RMI Layers



# RMI-based Application Development and Execution

## **Server Development:**

1. Design the interface for each service/object.
2. Implement the methods specified in the interface.
3. Registers objects with “remote object” registry (or Daemon)

## **Run time Execution:**

On the server:

3. Dynamically generate the stub.
4. Register the service by the name and location.

Client:

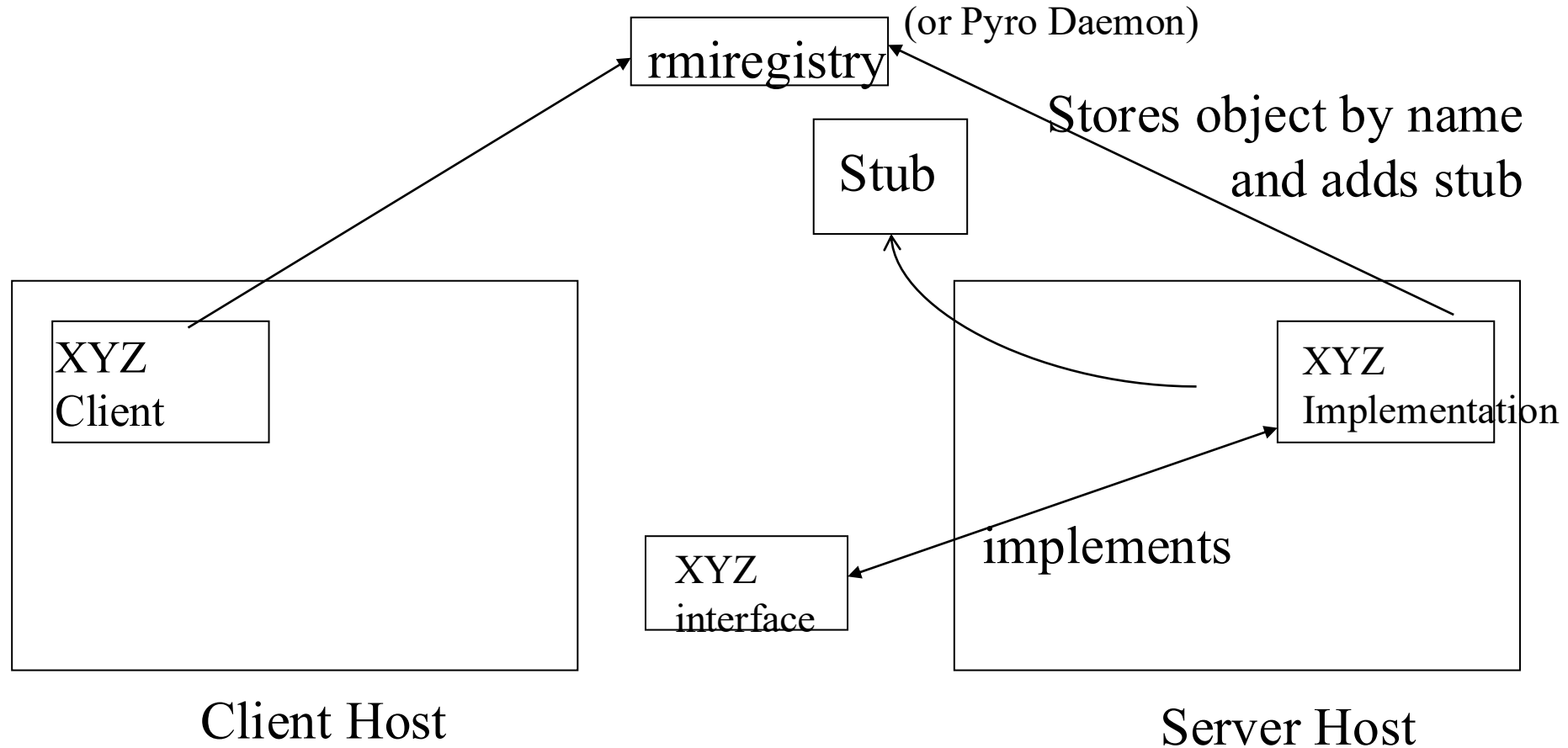
5. Look up remote reference on the registry (or daemon).
6. Uses normal method call syntax for remote methods
7. Use the service in an application.

# Object Broker: **Registry** (Java) or **Daemon** (Python)

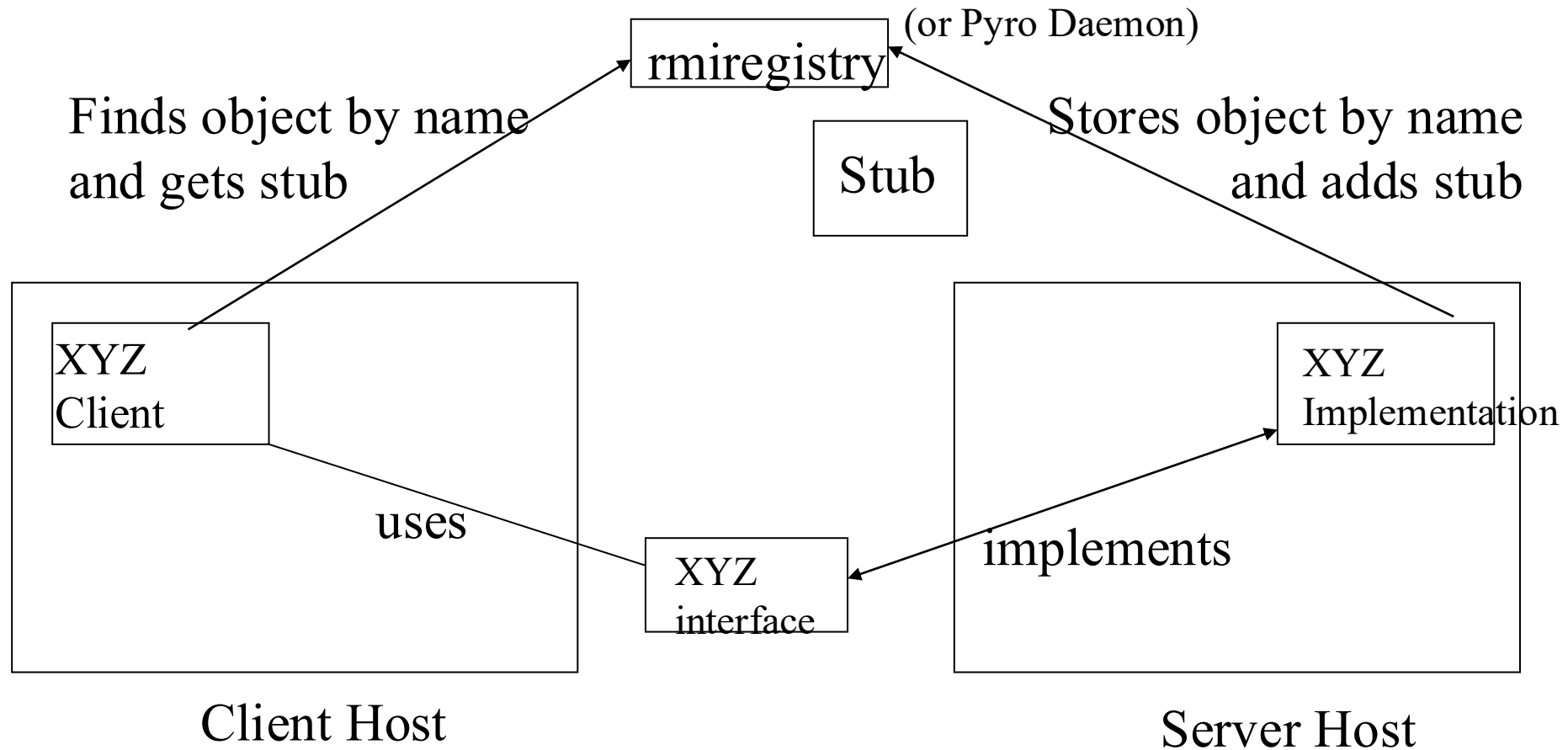
- A registry or a daemon is a running process on a host machine
- Maintain the names of remote objects and help look up these objects
  - **Servers** can register their objects
  - **Clients** can find server objects (by name) and obtain a remote reference
  - **Clients** can obtain a stub for a remote object, and request a list of remote objects from the registry



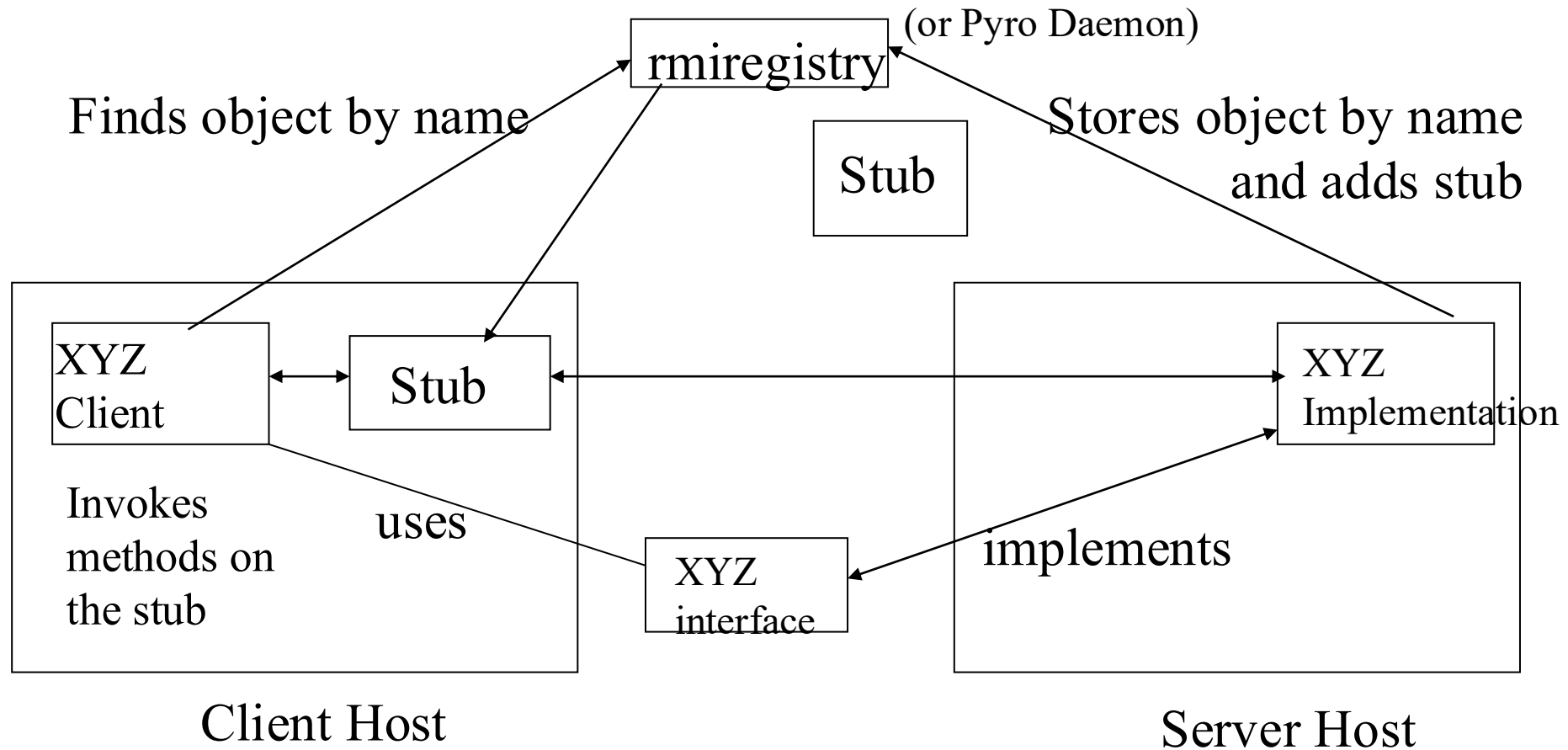
# Compile and Register Commands



# Compile and Register Commands



# Compile and Register Commands



# Properties of OOM

- Follow the object-oriented programming model
  - objects, methods, interfaces, encapsulation (public/private objects), and exception handling
  - Pass local objects by value and remote objects by references
- Synchronous request/reply interaction
  - Same as RPC

# Properties of OOM

- Location Transparency
  - Object request broker maps object references to physical locations. See this Python RMI (Pyro) example:  
**Remote = Pyro4.Proxy(uri)**  
**Response = remote.hello('Hello')**  
Uri: uniform resource identifier
- Services comprising multiple servers are easier to build
  - All services can be acquired through the object request broker
  - Geographical complexity and changes in services are hidden

# Example 1 – Python RMI Server

```
import Pyro4

class MyServer:

 @Pyro4.expose
 def say_hello(self):
 return "Hello, client!"

daemon = Pyro4.Daemon()

uri = daemon.register(MyServer)

print("Server URI:", uri)

daemon.requestLoop()
```

} remote  
function

} Start a remote  
server

← print out server  
placeholder

# Example 1 – Python RMI Client

```
import Pyro4

uri = "PYRO:obj_9be5165b05a34f9a86497301208e7227@localhost:14571"

my_server = Pyro4.Proxy(uri)
print(my_server.say_hello())
```

↑ use a remote  
server function

↑ generated by server  
each time

↑ connect a server object

# Example 2 – Python RMI Server (with nameserver, i.e., RMI registry)

```
python3 -m Pyro4.naming (default port = 9090) ← name server

import Pyro4

class MyServer:

 @Pyro4.expose
 def say_hello1(self):
 return "Hello1, client!"

 @Pyro4.expose
 def say_hello2(self):
 return "Hello2, client!"

daemon = Pyro4.Daemon()
uri = daemon.register(MyServer)

ns = Pyro4.locateNS(host="localhost", port=9090)
ns.register("MyServer", uri)

print("Server URI:", uri)
daemon.requestLoop()
```

} create a remote server

} register this server to the nameserver



## Example 2 – Python RMI Client

```
import Pyro4

ns = Pyro4.locateNS(host="localhost", port=9090)

uri = ns.lookup("MyServer")

my_server = Pyro4.Proxy(uri)

print(my_server.say_hello1())

print(my_server.say_hello2())
```

# Sample Runtime Screenshot

Command Prompt - python3 -m Pyro4.naming

```
Microsoft Windows [Version 10.0.19044.2486]
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\cgpro>python3 -m Pyro4.naming
Not starting broadcast server for localhost.
NS running on localhost:9090 (127.0.0.1)
Warning: HMAC key not set. Anyone can connect to this server!
URI = PYRO:Pyro.NameServer@localhost:9090
```

Command Prompt - python3 server.py

```
D:\00-2021\00-teaching\00-2022\00-ds\00-pyro>python3 server.py
Server URI: PYRO:obj_71f58f968bc3437da273d5b48610a697@localhost:5331
```

Command Prompt

```
D:\00-2021\00-teaching\00-2022\00-ds\00-pyro>python3 client.py
Hello1, client!
Hello2, client!

D:\00-2021\00-teaching\00-2022\00-ds\00-pyro>
```

# Pyro Server with Exceptional Handling

```
import Pyro4

class Calculator(object):
 @Pyro4.expose
 def add(self, x, y):
 return x + y

 @Pyro4.expose
 def subtract(self, x, y):
 return x - y

 @Pyro4.expose
 def multiply(self, x, y):
 return x * y

 @Pyro4.expose
 def divide(self, x, y):
 try:
 result = x / y
 except ZeroDivisionError:
 raise ValueError("Cannot divide by zero")
 return result

daemon = Pyro4.Daemon()
calc = Calculator()
uri = daemon.register(calc)

print("Server ready. Object uri =", uri)
daemon.requestLoop()
```

# Corresponding Example Pyro Client

```
import Pyro4

uri = input("Enter the URI of the calculator object: ").strip()
calc = Pyro4.Proxy(uri)

while True:
 x = float(input("Enter x: "))
 y = float(input("Enter y: "))
 try:
 print("Result:", calc.add(x, y))
 print("Result:", calc.subtract(x, y))
 print("Result:", calc.multiply(x, y))
 print("Result:", calc.divide(x, y))
 except Exception as e:
 print(e)
```

# References

- Chapter 4 (for RPC) and Chapter 2 (for RMI) of  
*Distributed Systems, 4th Edition, Maarten van Steen and Andrew Tannenbaum.*
- Chapter 5 of  
*Distributed systems : concepts and design / George Coulouris ... [et al.], Addison-Wesley (Pearson), c2012.*



# Middleware for system integration

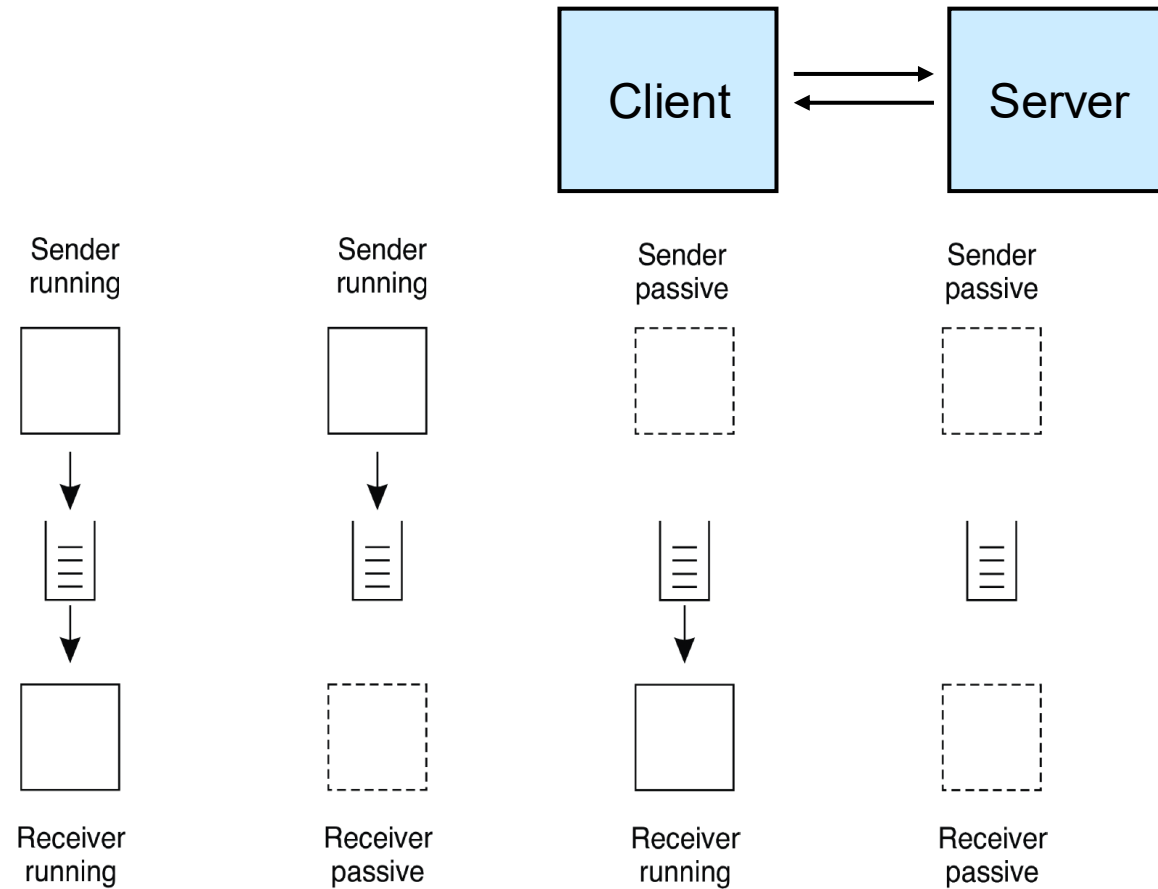
- RPC, Java RMI (and Python PYRO):
  - Closely follow the traditional program development process that creates an application based on 1) program libraries integration and 2) parameter passing
  - Do not require developers to deal with socket programming when implementing programs that support remote communication
  - Have a performance issue due to synchronous communication

A glass bottle with a cork and a rolled-up message inside, floating on water. The bottle is tilted, and the message is visible through the glass. The water is blue and has ripples around the bottle.

# Message Oriented Middleware (MOM)

# Message-Oriented Middleware (MOM)

- **Aim: We need something more loosely coupled**
- Communication using **messages**
- Messages stored in **message queues**

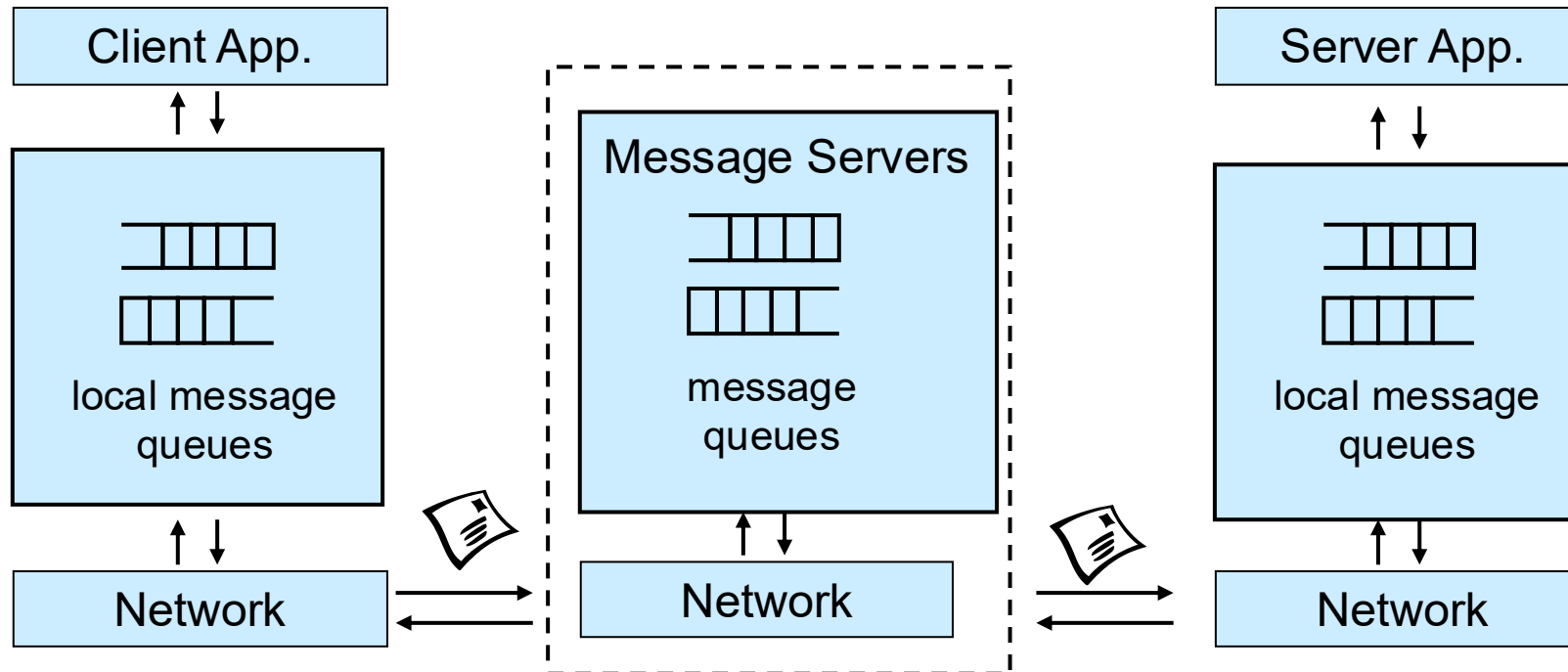


Source: Steen & Tannenbaum textbook



# Message-Oriented Middleware (MOM)

- Asynchronous persistent communication using middleware-level queues. Queues correspond to buffers at communication servers.
- **Message servers/Brokers** decouple client and server
- Various assumptions about **message content**



# Properties of MOM

- **Asynchronous** interaction
  - Client and server are only **loosely coupled** (vs RMI)
  - Messages are queued
  - Good for application integration
- Support for **reliable** delivery service
  - Keep queues in persistent storage
- Processing of messages by intermediate message server(s)
  - May perform filtering, transforming, and logging to detect similarities, prioritise request messages, etc.
  - Networks of message servers

## Ex: Java Message Service (JMS)

- Application Programming Interface (API) **specification** to access MOM implementations
- Two modes of operation:
  - **Point-to-point**
    - one-to-one communication
  - **Publish/Subscribe**
    - one-to-many communication
- **JMS Server** implements the JMS API
- JMS Clients connect to JMS servers
- Java objects can be serialised to JMS messages
- <https://www.oracle.com/technical-resources/articles/java/intro-java-message-service.html>

Ref: <http://docs.oracle.com/javaee/6/tutorial/doc/bncdq.html> (Java)  
<https://www.rabbitmq.com/tutorials/tutorial-one-python.html> (Python)

# Preparation: Run RabbitMQ Message Server

- To run the RabbitMQ message server, you first need to have RabbitMQ installed on your machine:
  - **Windows:** Download and install the RabbitMQ installer.
  - **macOS:** brew install rabbitmq.
  - **Linux:** You can use the package manager of your distribution to install RabbitMQ, e.g., on Ubuntu and Debian you can use the command `sudo apt-get install rabbitmq-server`.
- Start the RabbitMQ server by running the command **rabbitmq-server** in the command prompt/terminal.
- Can also use the management plugin to monitor the queue and check the status of the rabbitmq server by running the command **rabbitmq-plugins** enable **rabbitmq\_management** in the command prompt/terminal.

# Preparation: Run RabbitMQ Message Server

- Once the server is running, run your RabbitMQ server and client programs as separate processes to communicate via message queues.
- Note: you need to have **erlang** installed in your system to run rabbitmq, erlang is the client library.
- <https://www.rabbitmq.com/tutorials/tutorial-one-python.html>
- <https://hevodata.com/learn/rabbitmq-queue/>

# Example – RabbitMQ Server

```
import pika

Connect to RabbitMQ server
connection = pika.BlockingConnection(pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

Declare the two queues
channel.queue_declare(queue='reverse')
channel.queue_declare(queue='uppercase')

Define the callback function for the "reverse" queue
def reverse_callback(ch, method, properties, body):
 reversed_message = body.decode('utf-8')[::-1]
 ch.basic_publish(exchange='',
 routing_key=properties.reply_to,
 properties=pika.BasicProperties(correlation_id = \
 properties.correlation_id),
 body=reversed_message)
 ch.basic_ack(delivery_tag = method.delivery_tag)

Define the callback function for the "uppercase" queue
def uppercase_callback(ch, method, properties, body):
 uppercase_message = body.decode('utf-8').upper()
 ch.basic_publish(exchange='',
 routing_key=properties.reply_to,
 properties=pika.BasicProperties(correlation_id = \
 properties.correlation_id),
 body=uppercase_message)
 ch.basic_ack(delivery_tag = method.delivery_tag)

Set up the consumer for the "reverse" queue
channel.basic_consume(queue='reverse',
 on_message_callback=reverse_callback)

Set up the consumer for the "uppercase" queue
channel.basic_consume(queue='uppercase',
 on_message_callback=uppercase_callback)

Start consuming messages
print(' [x] Waiting for messages. To exit press CTRL+C')
channel.start_consuming()
```

# Example – RabbitMQ Client

```
import pika
import uuid

Connect to RabbitMQ server
connection = pika.BlockingConnection(pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

Declare a temporary queue for receiving the result
result = channel.queue_declare(queue='', exclusive=True)
callback_queue = result.method.queue

Define the callback function for the result
def on_response(ch, method, props, body):
 if props.correlation_id == corr_id:
 print(" [x] Received: %r" % body)

Set up the consumer for the temporary queue
channel.basic_consume(queue=callback_queue,
 on_message_callback=on_response,
 auto_ack=True)

Generate a unique correlation ID for the request
corr_id = str(uuid.uuid4())

Prompt user for message and function choice
message = input("Enter a message: ")
function_choice = input("Choose a function (reverse or uppercase): ")
```

```
Send message to appropriate queue
if function_choice == "reverse":
 channel.basic_publish(exchange='',
 routing_key='reverse',
 properties=pika.BasicProperties(
 reply_to=callback_queue,
 correlation_id=corr_id,
),
 body=message)
elif function_choice == "uppercase":
 channel.basic_publish(exchange='',
 routing_key='uppercase',
 properties=pika.BasicProperties(
 reply_to=callback_queue,
 correlation_id=corr_id,
),
 body=message)
else:
 print("Invalid function choice.
 Please choose either 'reverse' or 'uppercase'.")

Wait for the result
print(' [x] Sent %r' % message)
while True:
 connection.process_data_events()
```

# **COMP2211**

## **Distributed Systems**

### **Topic 3: Web Services**

*Thiru Vasantam*



# This Topic

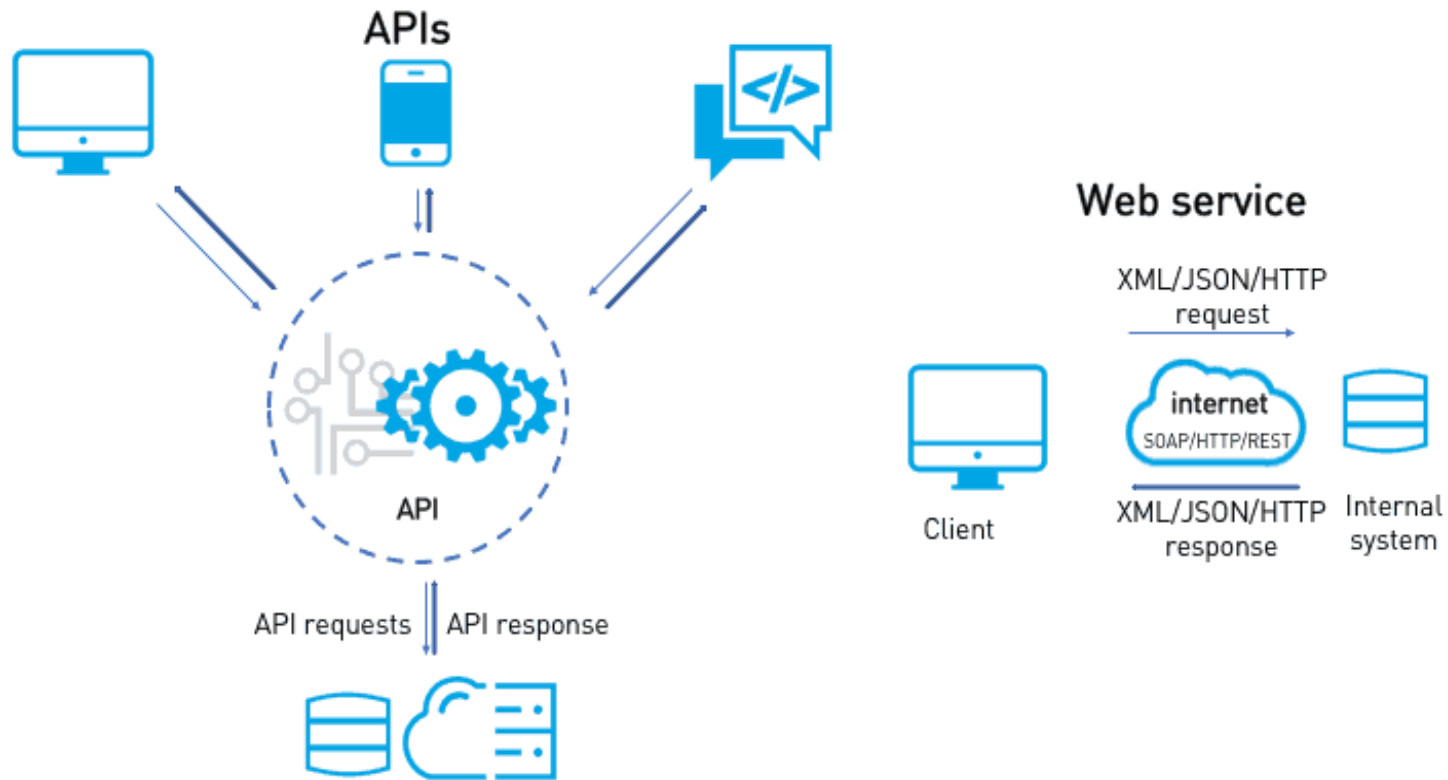
## Aim

Introduction to Web Services

Discuss some real-world distributed systems

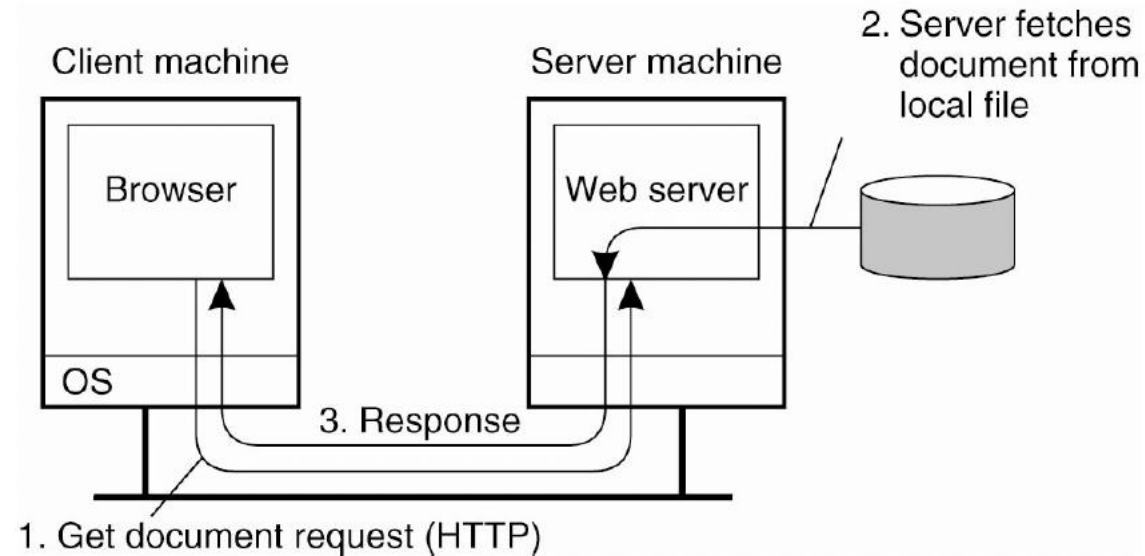
# Web Services

Web services provide a service interface that enables users to communicate with their servers over the Internet. The service interface describes the operations of web services.



# Client-Server Interaction

A file is accessed through Unique Resource Locator (URL)



Source: Steen & Tannenbaum textbook

# Reference

- **Chapter 4 (for MOM) and Chapters 2,3,7 (for Web Services) of**  
*Distributed Systems, 4th Edition, Maarten van Steen and Andrew Tannenbaum.*
- **Chapters 8 and 9 of**  
*Distributed systems : concepts and design / George Coulouris ... [et al.], Addison-Wesley (Pearson), c2012.*

